

The `lalign` Package

Jamie Fravel

Published as `lalign` v1.5 on July 6th 2026

`jamiefravel@me.com`

Abstract

This is a collection of macros that I developed while writing my dissertation. They are mostly focused on displaying, naming and referencing mathematical programming problems. The `lalign` environment macro formats math programs for legibility in both code and display. The named paragraph and sub-equation macros are useful in any document which defines and references multiple problems, algorithms or multi-line environments.

Contents

1	Installation	2
2	The <code>lalign</code> Environment	2
3	The <code>lregion</code> Environment	8
4	Named Paragraphs	9
5	Named Sub-equations	9
6	Sub-References	11
7	Combining Macros	12
8	Spacing Macros	13
9	Package Options	15
10	Known Bugs	16
11	Version History	17

1 Installation

Copy `lpsolve.sty` to the same directory as your main `.tex` file and add the command `\usepackage{lpsolve}` to your preamble. The package loads `IEEEtrantools`, `expl3` and `l3keys2e`. If available, it also loads `mathtools` and `stackengine`, with small internal fallbacks for the features used here. I recommend using these macros in conjunction with the `hyperref` package so that custom tags/references are clickable.

2 The `lpsolve` Environment

`lpsolve` is an environment macro built on the `IEEEeqnarray` environment from `IEEEtrantools`. It formats mathematical programs so that they are legible in code as well as in the final pdf. Its primary functions are to automatically format and position the ‘sense’ (maximize or minimize) and ‘subject to’ terms within a gutter to the left of the body in addition to aligning constraints and qualifiers.

The `lpsolve` environment takes two arguments—`<sense>` and `<objective>`—along with several optional keys:

- `vars` – subscript on the ‘sense’ term. Commonly used to state variables and ranges.
- `vars-style` – font style for the `vars` value. Defaults to `\scriptstyle` and may be changed during package load by `lp_vars_style`; see Section 9.
- `st` – text used for the ‘subject to’ term. Defaults to `s.t.` and may be changed during package load by `lp_st`.
- `flush` – controls the alignment behavior of the main constraint column. Takes options `l`, `c`, `r` and `a`. The default value of `a` can be changed during package load by `lp_flush` option.
- `row-sep` – controls the spacing between constraints. The default value of `0.5em` can be changed during package load by the `lp_row_sep` option.
- `objective-sep` – controls the spacing between objective function and the first constraint as added to `row-sep`. The default value of `0em` can be changed during package load by the `lp_objective_sep` option.
- `qualifier-sep` – controls the spacing between constraint body and the qualifier (for-all) term. The default value of `2em` can be changed during package load by the `lp_qualifier_sep` option.
- `gutter-sep` – controls the spacing between ‘sense’/‘subject to’ terms and the constraint body. The default value of `0.75em` can be changed during package load by the `lp_gutter_sep` option.
- `sense-shift` – adds a vertical shift to the ‘sense’ term.
- `st-shift` – adds a vertical shift to the ‘subject to’ term.

The `lpsolve*` environment is identical without printing equation reference tags unless explicitly called for by the `\tag` command.

```

\begin{lpalign}{Maximize}{\mathbf{c}^\top\mathbf{x}}
  A\mathbf{x} \&\le \mathbf{b} \quad \backslash\backslash
  \mathbf{x} \&\ge \mathbf{0}
\end{lpalign}

```

$$\text{Maximize } \mathbf{c}^\top \mathbf{x} \quad (1)$$

$$\text{s.t. } A\mathbf{x} \leq \mathbf{b} \quad (2)$$

$$\mathbf{x} \geq \mathbf{0} \quad (3)$$

Constraint Alignment

The environment defaults to `alignat`-style behavior; the constraint rows will be aligned at the ampersand `&`.

```

Ax+s \&= b+0 \backslash\backslash
x,s \&\ge 0

```

```

Ax+\&s = b+0 \backslash\backslash
\&x,s \ge 0

```

$$\max \quad \mathbf{c}^\top \mathbf{x}$$

$$\text{s.t. } A\mathbf{x} + \mathbf{s} = \mathbf{b} + \mathbf{0}$$

$$\mathbf{x}, \mathbf{s} \geq \mathbf{0}$$

$$\max \quad \mathbf{c}^\top \mathbf{x}$$

$$\text{s.t. } A\mathbf{x} + \mathbf{s} = \mathbf{b} + \mathbf{0}$$

$$\mathbf{x}, \mathbf{s} \geq \mathbf{0}$$

The `flush` argument can be used to adjust global alignment. Options are `l`, `c` and `r` in addition to its default behavior `a`.

<code>\begin{lpalign*}</code>	<code>\begin{lpalign*}</code>	<code>\begin{lpalign*}</code>
<code>[flush=l]</code>	<code>[flush=c]</code>	<code>[flush=r]</code>
<code>{max}{c^\top x}</code>	<code>{max}{c^\top x}</code>	<code>{max}{c^\top x}</code>
<code>Ax+s = b+0 \backslash\backslash</code>	<code>Ax+s = b+0 \backslash\backslash</code>	<code>Ax+s = b+0 \backslash\backslash</code>
<code>x,s \ge 0</code>	<code>x,s \ge 0</code>	<code>x,s \ge 0</code>
<code>\end{lpalign*}</code>	<code>\end{lpalign*}</code>	<code>\end{lpalign*}</code>

$$\begin{array}{lll}
\max & c^\top x & \max & c^\top x & \max & c^\top x \\
\text{s.t.} & Ax + s = b + 0 & \text{s.t.} & Ax + s = b + 0 & \text{s.t.} & Ax + s = b + 0 \\
& x, s \geq 0 & & x, s \geq 0 & & x, s \geq 0
\end{array}$$

Qualified Constraints

Qualified constraints are also possible. Another ampersand & will properly space and align a ‘for-all’ ($\forall$) statement.

```

\begin{lpalign*}{Maximize}{\sum_{j=1}^m c_j x_j}
\mathbf{a}_i^\top \mathbf{x} \leq b_i
& \forall i \in [n] \\
x_i \geq 0 & \forall i \in \{1, 2, \dots, n\}

```

$$\begin{array}{ll}
\text{Maximize} & \sum_{j=1}^m c_j x_j \\
\text{s.t.} & \mathbf{a}_i^\top \mathbf{x} \leq b_i \quad \forall i \in [n] \\
& x_i \geq 0 \quad \forall i \in \{1, 2, \dots, n\}
\end{array}$$

The qualifier column still works with a centered/flushed environment body.

Variable Ranges

Subscripts can be placed under the ‘sense’ term via the `vars` option.

```

\begin{lpalign*}[vars=\mathbf{x} \in \mathbb{R}_+]
{Maximize}{c_1 x_1 + \dots + c_m x_m}
A \mathbf{x} \leq \mathbf{b}
\end{lpalign*}

```

$$\begin{array}{ll}
\text{Maximize} & c_1 x_1 + \dots + c_m x_m \\
& \mathbf{x} \in \mathbb{R}_+ \\
\text{s.t.} & A\mathbf{x} \leq \mathbf{b}
\end{array}$$

Whitespace

The `sep` options can be used to modify the whitespace between elements. Their default values are mutable at package load.

```
\begin{lpalign}
[
objective-sep=1em,
qualifier-sep=3em,
gutter-sep=3em,
row-sep=1em,
]
{Maximize}{\sum_{j=1}^m c_j x_j}
\mathbf{a}_i^\top \mathbf{x} \leq b_i
& \forall i \in \llbracket n \rrbracket \quad \backslash\backslash
\mathbf{a}_i^\top \mathbf{x} \geq d_i
& \forall i \in \llbracket n \rrbracket \quad \backslash\backslash[3em]
x_i \geq 0 \quad \forall i \in \{1, 2, \dots, n\}
\end{lpalign}
```

$$\text{Maximize} \quad \sum_{j=1}^m c_j x_j \quad (4)$$

$$\text{s.t.} \quad \mathbf{a}_i^\top \mathbf{x} \leq b_i \quad \forall i \in \llbracket n \rrbracket \quad (5)$$

$$\mathbf{a}_i^\top \mathbf{x} \geq d_i \quad \forall i \in \llbracket n \rrbracket \quad (6)$$

$$x_i \geq 0 \quad \forall i \in \{1, 2, \dots, n\} \quad (7)$$

`objective-sep` is added to `row-sep` between the objective and first constraint rows. Remember that additional space can be added between individual rows (or lines of text) via an optional argument on `\backslash\backslash`.

Multiline Objectives

Very long `objective functions` can be expressed in multiple lines by supplying an explicit `aligned` environment from `amsmath`. This avoids overflow and keeps the objective aligned with the guttered ‘sense’ term. The same can be done within the `constraints`. Tags and qualifiers will be placed correctly.

```

\begin{lpalign}{Maximize}
  {\begin{aligned}
    c_1&x_1+c_2x_2+c_3x_3+c_4x_4 \\
    &+c_5x_5+\dots+c_mx_m
  \end{aligned}}
\end{aligned}}
\begin{aligned}
  a_{i1}&x_1+a_{i2}x_2+a_{i3}x_3 \\
  &+a_{i4}x_4+\dots+a_{im}x_m
\end{aligned}
&\leq b_i \quad \forall i \in \llbracket n \rrbracket
\\
\mathbf{x} &\geq \mathbf{0}
\end{aligned}
\end{lpalign}

```

$$\text{Maximize} \quad \begin{aligned} &c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 \\ &+ c_5x_5 + \dots + c_mx_m \end{aligned} \quad (8)$$

$$\text{s.t.} \quad \begin{aligned} &a_{i1}x_1 + a_{i2}x_2 + a_{i3}x_3 \\ &+ a_{i4}x_4 + \dots + a_{im}x_m \end{aligned} \leq b_i \quad \forall i \in \llbracket n \rrbracket \quad (9)$$

$$\mathbf{x} \geq \mathbf{0} \quad (10)$$

The ‘sense’ and ‘subject to’ terms can be freely shifted up or down. Other `amsmath`-compatible wrappers or environments can be nested within the objective but math mode may need to be manually re-engaged

```

\begin{lpalign}
  [ sense-shift=0.5\baselineskip,
    st-shift=0.5\baselineskip ]
  {Maximize}
  {\fbox{$\begin{aligned}
    c_1&x_1+c_2x_2+c_3x_3+c_4x_4 \\
    &+c_5x_5+\dots+c_mx_m
  \end{aligned}$}}
  ...
\end{lpalign}

```

$$\text{Maximize} \quad \boxed{\begin{aligned} &c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 \\ &+ c_5x_5 + \dots + c_mx_m \end{aligned}} \quad (11)$$

$$\text{s.t.} \quad \begin{aligned} &a_1x_1 + a_2x_2 + a_3x_3 \\ &+ a_4x_4 + \dots + a_mx_m \end{aligned} \leq b_i \quad \forall i \in \llbracket n \rrbracket \quad (12)$$

$$\mathbf{x} \geq \mathbf{0} \quad (13)$$

Note that, for some nested environments like `aligned` from `amsmath`, the optional argument `[t]` will align the sense term with the top row of the environment. Be careful when using this feature in the constraints as it will disrupt alignment with the remaining row.

Span Formatting

The `\span` command tells `IEEEeqnarray` (and therefore `lpalign`) to ignore a column for a particular row. This is useful for letting qualifiers wrap bulky constraints.

```
\begin{lpalign}{minimize}{c_1x_2+c_2x_2+\dots+c_nx_n}
&\sum_{j=1}^na_{ij}x_j \geq b_i &\forall i \in \llbracket m \rrbracket \\\
&\mathbf{a}_i^{\top} \mathbf{x} \geq b_i &\forall i \in \llbracket m \rrbracket \\\
&\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \geq \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \\
&\mathbf{A}\mathbf{x} \geq \mathbf{b} \\
\end{lpalign}
```

$$\text{minimize } c_1x_2 + c_2x_2 + \dots + c_nx_n \quad (14)$$

$$\text{s.t. } \sum_{j=1}^n a_{ij}x_j \geq b_i \quad \forall i \in \llbracket m \rrbracket \quad (15)$$

$$\mathbf{a}_i^{\top} \mathbf{x} \geq b_i \quad \forall i \in \llbracket m \rrbracket \quad (16)$$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \geq \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \quad (17)$$

$$\mathbf{A}\mathbf{x} \geq \mathbf{b} \quad (18)$$

An `\hfill`, two `\span`'s and one zero-width space will right-flush a row. Be careful when using `\span` alongside the `flush` option since there is one fewer column.

3 The `lpregion` Environment

The `lpregion` environment mirrors the formatting features of `lpalign` without objective, sense or s.t. terms for the purpose of describing regions only. It takes only a few optional keys:

- `flush` – controls the alignment behavior of the main constraint column. Takes options `l`, `c`, `r` and `a`. The default value of `a` can be changed during package load by `reg_flush` option.
- `row-sep` – controls the spacing between constraints. The default value of `0.5em` can be changed during package load by the `lp_row_sep` option.
- `qualifier-sep` – controls the spacing between constraint body and the qualifier (for-all) term. The default value of `2em` can be changed during package load by the `lp_qualifier_sep` option.

The `lpregion*` environment is identical without printing equation reference tags unless explicitly called for by the `\tag` command.

```
\begin{lpregion*}
&\sum_{j=1}^n a_{ij}x_j \geq b_i &\forall i \in \llbracket m \rrbracket \\
&\mathbf{a}_i^\top \mathbf{x} \geq b_i &\forall i \in \llbracket m \rrbracket \\
&\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \geq \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \\
&\mathbf{A} \mathbf{x} \geq \mathbf{b}
\end{lpregion*}
```

$$\sum_{j=1}^n a_{ij}x_j \geq b_i \quad \forall i \in \llbracket m \rrbracket \quad (19)$$

$$\mathbf{a}_i^\top \mathbf{x} \geq b_i \quad \forall i \in \llbracket m \rrbracket \quad (20)$$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \geq \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \quad (21)$$

$$\mathbf{A} \mathbf{x} \geq \mathbf{b} \quad (22)$$

4 Named Paragraphs

The `namedpara` command displays a paragraph marker that takes an abbreviation, which is useful for referencing specific problems or conventions.

```
\namedpara{Named Paragraph}[NPg]\label{npara1} \\  
Paragraph \ref{npara1} is a named paragraph.
```

Named Paragraph (NPg)

Paragraph [NPg](#) is a named paragraph.

Currently supported optional keys include

- `abv` – abbreviation used when referencing. A single argument with `no =` sign is also interpreted as this abbreviation.
- `indent` – indent distance. The default value of `2em` can be changed during package load by the `namedpara_indent` option; see Section 9.
- `before` – arbitrary code placed between the indent and the title.
- `after` – arbitrary code placed after the abbreviation.
- `flush` – centers or right-flushes the paragraph title via `center` or `right`.

```
\namedpara{Indented}[abv=IND, indent=6em]  
  
\namedpara{Surrounded}  
[abv=SUR, flush=center, before=$>>>$, after=$<<<$]  
  
\namedpara{Rightflushed}[abv=RIGHT, flush=right]
```

Indented (IND)

>>>Surrounded (SUR)<<<

Rightflushed (RIGHT)

5 Named Sub-equations

The `namedsubeqs` environment uses a given name as the first index in any enclosed equation reference tag. A second index is appended as in a traditional `subequations` environment.

```
\begin{namedsubeqs}{EQ}  
\begin{align}  
A\mathbf{x} = \mathbf{b} \quad \label{eq:1} \quad \\\end{align}
```

```

C\mathbf{y} = \mathbf{d} \quad \label{eq:2}
\end{align}
\end{namedsubeqs}

Equations \ref{eq:1} and \ref{eq:2} are equations.

```

$$Ax = b \quad (\text{EQ.a})$$

$$Cy = d \quad (\text{EQ.b})$$

Equations [EQ.a](#) and [EQ.b](#) are equations.

An optional argument allows you to change the style of the sub-tag. The currently supported keys are `style` for the second-index formatter (default `\alph`) and `punct` for the separator between the main tag and the sub-tag (default `.`).

```

\begin{namedsubeqs}{EQ}[style=\roman]

\begin{namedsubeqs}{EQ}[style=\arabic]

```

$Ax = b$	(EQ.i)	$Ax = b$	(EQ.1)
$Cy = d$	(EQ.ii)	$Cy = d$	(EQ.2)

Fonts can be changed using the appropriate switch commands.

```

\begin{namedsubeqs}{EQ}[style=\itshape\roman]

\begin{namedsubeqs}{EQ}[style=\scshape]

```

$Ax = b$	(EQ. <i>i</i>)	$Ax = b$	(EQ. A)
$Cy = d$	(EQ. ii)	$Cy = d$	(EQ. B)

The `punct` argument controls the separator between the main tag and the sub-tag.

```
\begin{namedsubeqs}{EQ}[style=\scshape\roman, punct=:]

\begin{namedsubeqs}{EQ}[style=\bfseries\arabic, punct=$\sim$]
```

$A\mathbf{x} = \mathbf{b}$	(EQ:I)	$A\mathbf{x} = \mathbf{b}$	(EQ~1)
$C\mathbf{y} = \mathbf{d}$	(EQ:II)	$C\mathbf{y} = \mathbf{d}$	(EQ~2)

A single optional argument which does not contain an = sign is interpreted as the `style` argument. The default `style` and `punct` values may also be changed during package load with `namedsubeqs_style` and `namedsubeqs_punct`; see Section 9.

```
\begin{namedsubeqs}{EQ}[\sffamily\Alph]

\begin{namedsubeqs}{EQ}[\bfseries\arabic]
```

$A\mathbf{x} = \mathbf{b}$	(EQ.A)	$A\mathbf{x} = \mathbf{b}$	(EQ.A)
$C\mathbf{y} = \mathbf{d}$	(EQ.B)	$C\mathbf{y} = \mathbf{d}$	(EQ.B)

6 Sub-References

Also available are `\lpsubref` and `\lpsubeqref`, which work like `\ref` and `\eqref` but reference only the secondary, sub-index part of a tag inside either `namedsubeqs` or a traditional `subequations` environment from the `amsmath` package.

```
\begin{namedsubeqs}{SQ}[style=\itshape\roman]\label{nq}
\begin{align}
A\mathbf{x} = \mathbf{b} \quad \label{nq:1} \\
C\mathbf{y} = \mathbf{d} \quad \label{nq:2}
\end{align}
\end{namedsubeqs}
```

Just like `\ref{nq:1}` and `\eqref{nq:2}`, we can reference an equation's sub-tag `\lpsubref{nq:1}` and `\lpsubeqref{nq:2}`. The environment itself may also be referenced `\ref{nq}`.

$$A\mathbf{x} = \mathbf{b} \quad (\text{SQ}.i)$$

$$C\mathbf{y} = \mathbf{d} \quad (\text{SQ}.ii)$$

Just like [SQ.i](#) and [\(SQ.ii\)](#), we can reference an equation's sub-tag [i](#) and [\(ii\)](#). The environment itself may also be referenced [SQ](#).

`\lpsubref` and `\lpsubeqref` work with ordinary `\label` inside both `namedsubeqs` and traditional `subequations` environments. Other environments are not officially supported.

```
\begin{subequations}\label{sq}
\renewcommand{\theequation}
{\theparentequation\textbar\arabic{equation}}
\begin{align}
A\mathbf{x} = \mathbf{b} \quad \label{sq:1}\backslash
C\mathbf{y} = \mathbf{d} \quad \label{sq:2}
\end{align}
\end{subequations}
```

We can also reference `\ref{sq}`, `\ref{sq:1}` and `\eqref{sq:2}`, or sub-reference `\lpsubref{sq:1}` and `\lpsubeqref{sq:2}` in a `\textsf{subequations}` environment.

$$A\mathbf{x} = \mathbf{b} \quad (23|1)$$

$$C\mathbf{y} = \mathbf{d} \quad (23|2)$$

We can also reference [23](#), [23|1](#) and [\(23|2\)](#), or sub-reference [1](#) and [\(2\)](#) in a `subequations` environment.

Supported punctuation currently includes the period (`.`), colon (`:`), semicolon (`;`), comma (`,`), hyphen (`-`), forward slash (`/`), and `\textbar` (`|`). Other punctuation has not been tested.

7 Combining Macros

Each of these macros may be combined for a nicely named, labeled, formatted and tagged mathematical programming problem.

Non-Linear Program (NLP)

Problem (NLP) is a nonlinear program. Equation (NLP.i) is the objective function. Equations (NLP.ii-iv) are the constraints. Non-negativity is enforced by constraint (iv).

8 Spacing Macros

Quick Spacing

Also included are font-relative versions of the common spacing commands `\quad` and `\qquad` and a few more for good measure:

- | | | | | | |
|-------------------|--|--|---------------|--|--|
| • \hspace = 0.5em | | | • \HSKP = 3em | | |
| • \Hspace = 1em | | | • \HSPK = 4em | | |
| • \HSpk = 2em | | | | | |

Negative versions of the above spacing macros:

- \htrg = -0.5em
- \Htrg = -1em
- \HTrg = -2em
- \HTRg = -3em
- \HTRG = -4em

Vertical versions of the above horizontal spacing macros:

- `\vskp` = 0.5em
- `\Vskp` = 1em
- `\VSKp` = 2em
- `\VSKp` = 3em
- `\VSKP` = 4em
- `\vtrg` = -0.5em
- `\Vtrg` = -1em
- `\VTrg` = -2em
- `\VTRg` = -3em
- `\VTRG` = -4em

These are, I would say, less reliable than using `\vspace` or `\vskip` correctly, but I find them more convenient.

Some horizontal spacing between these \HSKP words

Some vertical spacing between these \VSKp lines

Some negative-horizontal spacing between these \HTrg words

Some negative-vertical space between these \Vtrg lines, so that they overlap.

Some horizontal spacing between these words

Some vertical spacing between these lines

Some negative-horizontal spacing between these words

Some negative-vertical space between these lines, so that they overlap.

Delimiter Spacing

Some macros for tightening display-style operator spacing when the indexing is over-long. These preserve the usual subscript and superscript syntax while placing long limits in `\mathclap` boxes. They are meant for display math and are not generally useful inline.

- `\SUM` = $\sum$
- `\CUP` = $\bigcup$
- `\VEE` = $\bigvee$
- `\PROD` = $\prod$
- `\CAP` = $\bigcap$
- `\WEDGE` = $\bigwedge$

These operators now use the ordinary `_` and `^` syntax. This spacing is a personal preference; I think it makes things more consistent and legible. The example below is comically exaggerated; please use `\substack` to illuminate any *really* long indexing.

$X = \sum_{i \text{ in way too much indexing}} x_i$
 $X = \bigvee_{i \text{ in way too much indexing}} x_i$
 $X = \bigcap_{i \text{ in way to \ \ much indexing}} x_i$
 $X = \prod_{i=1,300,000}^{1,300,001} x_i$

$$\begin{array}{ll}
 X = \sum_{i \text{ in way too much indexing}} x_i & X = \bigwedge_{i \text{ in way too much indexing}} x_i \\
 X = \bigcap_{i \text{ in way to much indexing}} x_i & X = \prod_{i=1,300,000}^{1,300,001} x_i
 \end{array}$$

Evaluator Spacing

A few binary relations with extra space on either side. I like these for aligned equations or inequality constraints.

- $\backslash EQ$ = • $\backslash IN$ $\in$ • $\backslash NEQ$ $\neq$ • $\backslash NIN$ $\notin$
- $\backslash LS$ < • $\backslash LE$ $\leq$ • $\backslash SUB$ $\subset$ • $\backslash SEQ$ $\subseteq$
- $\backslash GS$ > • $\backslash GE$ $\geq$ • $\backslash SUP$ $\supset$ • $\backslash SPQ$ $\supseteq$

$Ax + s \backslash eq b \ \backslash$ $x, s \backslash ge 0$	$Ax + s \backslash EQ b \ \backslash$ $x, s \backslash GE 0$
$\max \ c^\top x$ s.t. $Ax + s = b + 0$ $x, s \geq 0$	$\max \ c^\top x$ s.t. $Ax + s = b + 0$ $x, s \geq 0$

The spacing defaults to 0.25em and is controlled by the package option `evaluator_spacing`; see Section 9.

9 Package Options

Default behaviors of many features can be changed with package-load options. Use the following keys in `\usepackage[<key>=<value>]{lpsalign}`. For package-

load style defaults, use style names rather than raw control sequences. Thus `lp_vars_style=scriptstyle` and `namedsubeqs_style=roman` are valid package options. For `lp_st`, bare spaces in `\usepackage[...]` are stripped by LaTeX before this package sees the option value. Use `lp_st={subject to}` or `lp_st=subject~to` instead.

`lp_row_sep` – controls the spacing between the rows. Defaults to `0.5em`. Section 2.

`lp_objective_sep` – controls additional spacing between objective function and the first constraint. Defaults to `0em`.

`lp_qualifier_sep` – controls the spacing between constraint body and the qualifier (for-all) term. Defaults to `2em`.

`lp_gutter_sep` – controls the spacing between ‘sense’/‘subject to’ terms and the constraint body within `lpalign`. Defaults to `0.75em`.

`lp_vars_style` – controls the default `vars-style` in `lpalign`. Defaults to `\scriptstyle`.

`lp_st` – controls the default text in the ‘subject to’ gutter cell for `lpalign`. Defaults to `s.t..`

`lp_flush` – controls the default alignment behavior of constraints in the `lpalign` environment. Defaults to `a` but also accepts `l`, `c` and `r`.

`reg_flush` – controls the default alignment behavior of constraints in the `lpre` environment. Defaults to `a` but also accepts `l`, `c` and `r`. Section 3.

`namedpara_indent` – controls the indent of named paragraph markers. Defaults to `2em`. Section 4.

`namedparabefore` – controls the default material printed before a named paragraph abbreviation. Defaults to empty. Section 4.

`namedparaafter` – controls the default material printed after a named paragraph abbreviation. Defaults to empty.

`namedparaflush` – controls the default alignment of named paragraph markers. Defaults to `left`.

`namedsubeqs_style` – controls the default sub-index style used by `namedsubeqs`. Defaults to `\alph`. Section 6.

`namedsubeqs_punct` – controls the default punctuation between the named parent tag and the sub-index in `namedsubeqs`. Defaults to `..`

`evaluator_spacing` – controls the padding on either side of the spaced evaluators. Defaults to `0.25em`. Section 8.

10 Known Bugs

- Very long objective functions may overrun the page. For now, use the embedded `aligned` environment to display a long function on multiple lines.
- ☒ Spacing macros `\quad` and `\qqquad` overwrote default, absolute behavior with font-relative spacing. Fixed by renaming macros to `\Hskip` and `\HSkip`; v1.1.

- ☒ The package depended on `makecell` and `vcell` without using them. Fixed by removing those unused dependencies; v1.2.
- ☒ The `namedsubeqs` environment assigned `\currentlabel` without switching to `\makeatletter` scope. Fixed internally; v1.2.
- ☒ The `\lpsubref` and `\lpsubeqref` macros failed outright when `hyperref` was not loaded. Fixed by falling back to plain text references when `hyperref` is absent; v1.2.
- ☒ When `hyperref` was loaded, `\sublabel` could produce duplicate equation destinations or occasionally link to the wrong tag. Fixed by assigning dedicated `hyperref`-facing equation identifiers and by removing a redundant equation-counter decrement at the end of `namedsubeqs`; test branch after v1.2.
- ☒ Ordinary `\label` now works for `\lpsubref` and `\lpsubeqref` inside both `namedsubeqs` and traditional `subequations`. Fixed on the test branch after v1.2.
- ☒ The public commands `\subref` and `\subeqref` conflicted with packages such as `subcaption`. Fixed by renaming the package commands to `\lpsubref` and `\lpsubeqref`; v1.3.
- ☒ The objective function did not contribute correctly to the width of the environment. Fixed in v1.4 final.

11 Version History

- v1.0 Dec. 20, 2025. Initial distribution. Includes `lalign` and `namedsubeqs` environments along with macros for named paragraphs and spacing.
- v1.1 Jan. 30, 2026. Bug fix. Renamed quick spacing macros to maintain compatibility with standard $\TeX$. Added negative, vertical and negative-vertical versions. Added summation spacing macros. Added clause alignment arguments to the `lalign` environment.
- v1.2 Jun. 25, 2026. Bug fix. Removed unused package dependencies, fixed the internal `\currentlabel` handling in `namedsubeqs`, added a no-`hyperref` fallback for `\lpsubref` and `\lpsubeqref`, added `\lpsubref` and `\lpsubeqref` support for the traditional `subequations` environment from `amsmath`.
- v1.3 Jun. 26, 2026. Feature and compatibility update. Added `\namedpara` with key-value options, namespaced the sub-equation reference commands as `\lpsubref` and `\lpsubeqref`, and hardened parent-label handling for both `namedsubeqs` and traditional `subequations`.
- v1.4 Jun. 28, 2026. Layout and defaults update. Reworked `lalign` so the objective contributes correctly to the environment width, ordinary rows no longer need trailing `&` placeholders, package-load defaults are now available for `objective-sep`, `qualifier-sep` and `gutter-sep`, and the operator helpers now preserve standard `_/^` syntax.
- v1.5 Jul. 6, 2026. Parser version of `lalign` fixes s.t. alignment.